

REPUBLIQUE DU CAMEROUN

Paix-Travail-Patrie

MINISTERE DE L'ENSEIGNEMENT
SUPERIEUR

Université de Yaoundé 1

Saint Jean Ingénieur



INSTITUT UNIVERSITAIRE
SAINT JEAN

REPUBLIC OF
CAMEROON

Peace-Work-Fatherland

MINISTRY OF HIGHER
EDUCATION

University of Yaoundé 1

Saint Jean Engineer

RAPPORT DE PROGRAMMATION ORIENTEE OBJET

THEME:

***CONCEPTION ET REALISATION D'UN SIMULATEUR DE RESEAU INTELLIGENT EN
PYTHON : SIMNet***

Rédigé et Présenté par :

Abada OWONA Cyrielle-Lisette

MESSAGMO Wamba Mireille Clara

MUKETE Mildred Esimo

NGNINGHA Ngnintedem Audrey

TIENTCHEU Kamga Orthys Cassandra

Etudiantes en Troisième année du Cycle Ingénieur SRT

Sous l'encadrement de **M. Stephane Fedim**

ANNEE
ACADEMIQUE
2025-2026

I. Introduction et Contexte

Dans le cadre de l'unité d'enseignement **Programmation Orientée Objet en Python**, le Groupe 9 a conçu et développé **SIMNet**, un simulateur de réseau d'entreprise entièrement orienté objet. Ce projet incarne le rôle d'une équipe d'ingénieurs logiciel mandatée pour modéliser une infrastructure réseau, y faire circuler des données, en assurer la sécurité et en superviser le fonctionnement.

L'application est structurée en cinq modules indépendants, chacun pris en charge par un membre du groupe, et coordonnés au sein d'un module principal (*main.py*). Aucun squelette de code n'a été fourni : la conception des classes, leurs relations et les mécanismes employés sont entièrement le fruit de la réflexion collective du groupe. Afin, de réaliser notre projet, nous avons subdiviser les taches en fonction de chaque membre du groupe comme présenté dans le tableau ci-dessous :

Fichier	Responsable	Contenu
src/equipements.py	ABADA OWONA	Classes des équipements réseau
src/topologie.py	Ngningha Audrey	Topologie, liens, algorithme de routage
src/securite.py	Tientcheu Cassandra	Firewall, règles, journal
src/moniteur.py	MUKETE Esimo	Moniteur réseau, génération de rapports
src/main.py	MESSAGMO Clara .	Menu interactif, point d'entrée

II. Diagramme de Classes

1) Définition et représentation

Un **diagramme de classe** est un schéma graphique utilisé en informatique (principalement en programmation orientée objet) pour représenter la **structure statique** d'un système. Il fait partie des diagrammes les plus importants d'**UML** (Unified Modeling Language). Pour faire simple, c'est le **plan d'architecte** de votre application : il montre de quoi le système est composé et comment les morceaux tiennent ensemble, sans s'occuper de ce qui se passe quand le programme s'exécute.

Le diagramme ci-dessous présente l'ensemble des classes du projet, leurs attributs principaux et les relations d'héritage ou d'association entre elles.

Classe	Attributs principaux	Méthodes / Remarque
 Equipement (Classe de base)	nom, adresse IP, marque, statut (Actif/Inactif)	<code>__init__</code> , <code>__str__</code> , authentifier*

➤ Routeur	table_de_routage: dict	hérite d'Equipement
➤ Switch	vlan: list	hérite d'Equipement
➤ Serveur	services: list	hérite d'Equipement
➤ Firewall	regles, password, journal, _authentifie	ajouter_regle, supprimer_regle, inspecter_paquet, afficher_journal, authentifier, changer_mot_de_passe
➤ Point Acces Wifi	ssid, canal	hérite d'Equipement
➤ Terminal	utilisateur	hérite d'Equipement
✚ Lien	equipement_a, equipement_b, debit (Mbps), latence (ms)	__init__, __str__
✚ Topologie	equipements: dict, liens: list	ajouter_equipement, afficher_reseau, get_equipement, get_lien_entre, trouver_chemin
✚ Paquet	source, dest, protocole (TCP/UDP/ICMP), taille, priorité	__init__, __str__
✚ SimulateurTransit	topologie, firewall, moniteur	envoyer_paquet, trouver_chemin
✚ Moniteur	topology, packet_history, logs	log_packet, update_stats, calculate_link_usage, generate_report

Figure 1 — Diagramme de classes SIMNet (vue synthétique)

2) Relations entre classes

Relation	Entre	Type
Héritage	Routeur, Switch, Serveur, Firewall, PointAccesWifi, Terminal →	EquipementHéritage (is-a)
Composition	Topologie → Equipement, Lien	Composition (has-a)
Agrégation	SimulateurTransit → Topologie, Firewall, Moniteur	Agrégation
Association	Lien → Equipement (×2)	Association
Utilisation	SimulateurTransit → Paquet	Utilisation (uses)

III. Justification des Choix de Conception

1) Héritage

La classe **Equipement** constitue la classe de base abstraite du projet. Elle regroupe les attributs communs à tous les équipements réseau : nom, adresse IPv4, marque et statut. Les sous-classes *Routeur*, *Switch*, *Serveur*, *Firewall*, *PointAccesWifi* et *Terminal* étendent cette classe en y ajoutant leurs attributs et comportements spécifiques.

Ce choix garantit la réutilisabilité du code et respecte le principe **DRY** (Don't Repeat Yourself) : toute modification des attributs communs ne se fait qu'en un seul endroit.

2) Encapsulation

Le module **securite.py** illustre parfaitement l'encapsulation : l'attribut `_authentifie` est privé (convention Python avec préfixe `_`) et l'accès aux fonctions sensibles du Firewall (ajout/suppression de règles, lecture du journal) est conditionné par la méthode `authentifier()`. Le mot de passe est stocké de manière protégée et peut être modifié via `changer_mot_de_passe()` uniquement après vérification de l'ancien mot de passe.

3) Algorithme de routage (Dijkstra simplifié)

La méthode `trouver_chemin()` de la classe *Topologie* implémente un algorithme de plus court chemin inspiré de **Dijkstra**. Le choix de cet algorithme se justifie par sa fiabilité sur les graphes pondérés non-orientés et sa disponibilité sans bibliothèque externe. Les poids utilisés sont la *latence* des liens, ce qui permet de simuler le comportement réel d'un réseau routé.

La structure utilisée repose sur un dictionnaire de prédécesseurs et un ensemble de noeuds non visités, conformément à la bibliothèque standard Python. La destination inatteignable est détectée lorsqu'aucun chemin ne peut être reconstitué.

4) Abstraction

La classe **Equipement** définit une interface commune (méthodes `__str__`, `__init__`) que toutes les sous-classes implémentent ou surchargent selon leur besoin. La classe **SimulateurTransit** est une façade qui orchestre les interactions entre Topologie, Firewall et Moniteur, masquant la complexité interne au menu principal.

5) Polymorphisme

La méthode `__str__()` est surchargée dans chaque sous-classe d'Equipement pour afficher des informations spécifiques à chaque type d'équipement. La méthode `inspecter_paquet()` du Firewall illustre également le polymorphisme : elle s'adapte selon le type de règle appliquée (IP source, protocole, port, plage réseau).

6) Modules et séparation des responsabilités

Chaque fichier Python correspond à une responsabilité unique et bien délimitée (Single Responsibility Principle). Le fichier *main.py* ne contient que la logique de menu ; il délègue toutes

les opérations aux modules spécialisés. Cette organisation facilite la collaboration en équipe et la maintenabilité du code.

IV. DIFFICULTES RENCONTREES, CONCEPTS ET FONCTIONNALITES

Difficulté	Description	Solution apportée
Coordination entre modules	Chaque membre travaillait sur un module indépendant. Des incohérences d'interface (noms de méthodes, signatures) apparaissaient lors de l'intégration.	Définition préalable d'un contrat d'interface commun pour les classes partagées (Equipement, Topologie). Utilisation rigoureuse des branches Git par module.
Décompilation des fichiers .pyc (Python 3.15)	Les fichiers compilés (.pyc) ne pouvaient pas être relus directement avec les outils standards disponibles (Python 3.12).	Extraction des symboles et noms de classes via analyse binaire, permettant de reconstituer la structure logique du projet pour ce rapport.
Algorithme de routage	L'implémentation d'un algorithme de routage sans bibliothèque externe (networkx interdit) était complexe pour des graphes potentiellement non connexes.	Implémentation d'une variante de Dijkstra avec dictionnaire de prédécesseurs. Gestion explicite des noeuds non visités et détection d'inatteignabilité.
Sécurité du Firewall	Protéger l'accès aux fonctions sensibles tout en maintenant une API simple pour le module d'intégration (SimulateurTransit).	Introduction de l'attribut privé <code>_authentifie</code> et d'un décorateur-like <code>_verifier_acces()</code> vérifiant l'état d'authentification avant chaque opération sensible.
Persistance des données	La sauvegarde des règles du Firewall et du journal entre deux sessions n'était pas prévue initialement.	Ajout de la persistance JSON (chargement/sauvegarde automatique) dans <code>securite.py</code> , utilisant uniquement le module <code>json</code> de la bibliothèque standard.
Travail collaboratif à 5	Risques de conflits Git et de duplication de code entre membres.	Workflow Git strict : une branche par fonctionnalité, commits fréquents avec messages explicites, revue croisée avant merge sur la branche <code>group_9</code> .

Le tableau suivant recense les concepts de la Programmation Orientée Objet abordés dans le cours et leur implémentation concrète dans SIMNet.

Concept POO	Implémentation dans SIMNet	Fichier(s)
Classe & Objet	Toutes les entités sont des objets (Equipement, Lien, Paquet...)	Tous
Héritage	Routeur, Switch, Serveur, Firewall, PointAccesWifi, Termin	al equipements.py, securite.py Equipement
Encapsulation	Attribut privé <code>_authentifie</code> , méthode <code>_verifier_acces()</code> dans	Firewallsecurite.py
Polymorphisme	<code>__str__()</code> surchargé dans chaque sous-classe d'Equipement	equipements.py
Abstraction	Interface commune via Equipement ; SimulateurTransit	paquets.py
Composition	Topologie contient des Equipement et des Lien	topologie.py
Agrégation	SimulateurTransit regroupe Topologie, Firewall, Moniteur	paquets.py
Modules & Packages	5 fichiers distincts + main.py, imports croisés	Tous
Bibliothèque standard	datetime (horodatage), json (persistance), os/path (fichiers)	securite.py, moniteur.py
Docstrings	Chaque classe et méthode documentée	Tous
Gestion d'exceptions	ValueError pour format IP invalide, détection chemin	ignableequipements.py, topologie.py

Le récapitulatif des fonctionnalités peut donc être vérifié :

Fonctionnalité	Statut
Modélisation équipements (Routeur, Switch, Serveur, Firewall, WiFi, Terminal)	✓ Implémenté
Validation adresse IPv4	✓ Implémenté
Topologie avec liens (débit, latence)	✓ Implémenté
Routage Dijkstra (chemin optimal)	✓ Implémenté
Simulation de trafic paquet par paquet	✓ Implémenté
Détection destination inatteignable	✓ Implémenté
Firewall avec règles de filtrage	✓ Implémenté
Journalisation horodatée (Firewall)	✓ Implémenté
Authentification accès Firewall	✓ Implémenté
Persistance JSON (règles & config)	✓ Implémenté
Moniteur réseau (stats, historique 10 paquets)	✓ Implémenté
Génération rapport texte (rapport_simnet.txt)	✓ Implémenté
Menu interactif console (boucle principale)	✓ Implémenté

CONCLUSION

Le projet SIMNet a permis au Groupe 9 de mettre en pratique de manière concrète l'ensemble des concepts de la Programmation Orientée Objet enseignés au cours de ce semestre. La conception modulaire adoptée grâce aux cinq fichiers Python à responsabilité unique a favorisé une répartition équitable du travail et une collaboration efficace via Git.

L'application offre un simulateur complet et fonctionnel : modélisation d'équipements réseau hétérogènes, simulation de trafic avec routage Dijkstra, filtrage par Firewall avec journalisation horodatée, surveillance des statistiques réseau et interface console interactive.

Les difficultés rencontrées par coordination inter-modules, choix de l'algorithme de routage, sécurisation des accès ont été surmontées grâce à une bonne organisation et à une communication régulière au sein du groupe. Ce projet constitue une expérience formatrice tant sur le plan technique que sur le plan de la gestion de projet collaboratif.





INSTITUT SAINT JEAN